

Лабораторная работа №7

«Синтаксис команд. Стандартные и пользовательские функции, функции обработки массивов в Scilab»

Цель работы: получить навыки работы с встроенными и пользовательскими функциями.

Все функции, используемые в Scilab, можно разделить на два класса: встроенные и пользовательские.

В общем виде обращение к функции в Scilab имеет вид:

имя_переменной = имя_функции(переменная1 [, переменная2, ...]),

где имя_переменной - переменная, в которую будут записаны результаты работы функции (этот параметр может отсутствовать, тогда значение, вычисленное функцией будет присвоено системной переменной ans);

имя_функции - имя встроенной или пользовательской функции; переменная1, переменная2, ... - список аргументов функции.

Функция, как правило, предназначена для неоднократного использования, она имеет входные параметры и не выполняется без их предварительного задания. Рассмотрим несколько способов создания функций в Scilab.

Первый способ - это применение оператора `deff`, который в общем виде можно записать так:

`deff('[имя1,...,имяN]=имя_функции(переменная_1,...,переменная_M)', 'имя1=выражение1;...;имяN=выражениеN')`, где `имя1,...,имяN` - список выходных параметров, то есть переменных, которым будет присвоен конечный результат вычислений (параметров может быть от 1 до N);

Все имена переменных внутри функции, а так же имена из списка входных и выходных параметров воспринимаются системой как локальные, то есть эти переменные считаются определенными только внутри функции.

Вообще говоря, функции в Scilab играют роль подпрограмм. Поэтому

целесообразно набирать их тексты в редакторе и сохранять в виде отдельных файлов. Причем имя файла должно обязательно совпадать с именем функции. Расширение файлам-функциям обычно присваивают `sci` или `sce`.

Обращение к функции осуществляется так же, как и к любой другой встроенной функции системы, то есть из командной строки.

Программный режим достаточно удобен, так как он позволяет сохранить разработанный вычислительный алгоритм в виде файла и повторять его при других исходных данных в других сессиях. Кроме обращений к функциям и операторов присваивания, в программных файлах могут использоваться операторы языка программирования Scilab (язык программирования Scilab ещё называют `sci`-языком).

Функция `x_dialog` выводит на экран диалоговое окно с именем `title`, после чего пользователь может щелкнуть ОК и тогда `stroka` вернется в качестве результата в переменную `x`, либо ввести новое значение вместо `stroka`, которое и вернется в качестве результата в переменную `x` [5].

Функция `input` преобразовывает введенное значение к числовому типу данных, а функция `x_dialog` возвращает строковое значение. Поэтому при использовании функции `x_dialog` для ввода числовых значений, возвращаемую ею строку следует преобразовать в число с помощью функции `evstr`. Поэтому можно предложить следующую форму использования функции `x_dialog` для ввода числовых значений

Задание 1. Вычислить значения функции во всех точках заданного интервала с использованием стандартных и пользовательских функций. Для вычислений необходимо написать скрипт во встроенном редакторе Scipad. Скриншот варианта представлен на рисунке 1.

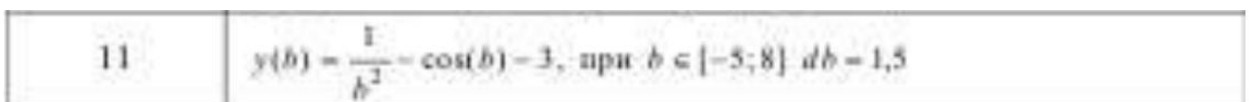
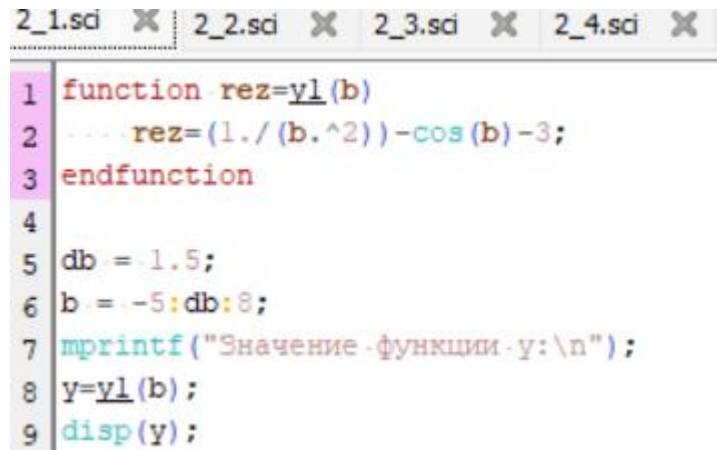


Рисунок 1 – Скриншот варианта к первому заданию

Создаём функцию, которая высчитывает значение при каждом `x`. Выводим значение функции с помощи команды `disp`. Результат работы

представлен на рисунке 2.



```
2_1.sci X 2_2.sci X 2_3.sci X 2_4.sci X
1 function rez=y1(b)
2 ... rez=(1./(b.^2))-cos(b)-3;
3 endfunction
4
5 db=-1.5;
6 b=-5:db:8;
7 mprintf("Значение функции y:\n");
8 y=y1(b);
9 disp(y);
```

Рисунок 2 – Скриншот кода к заданию 1

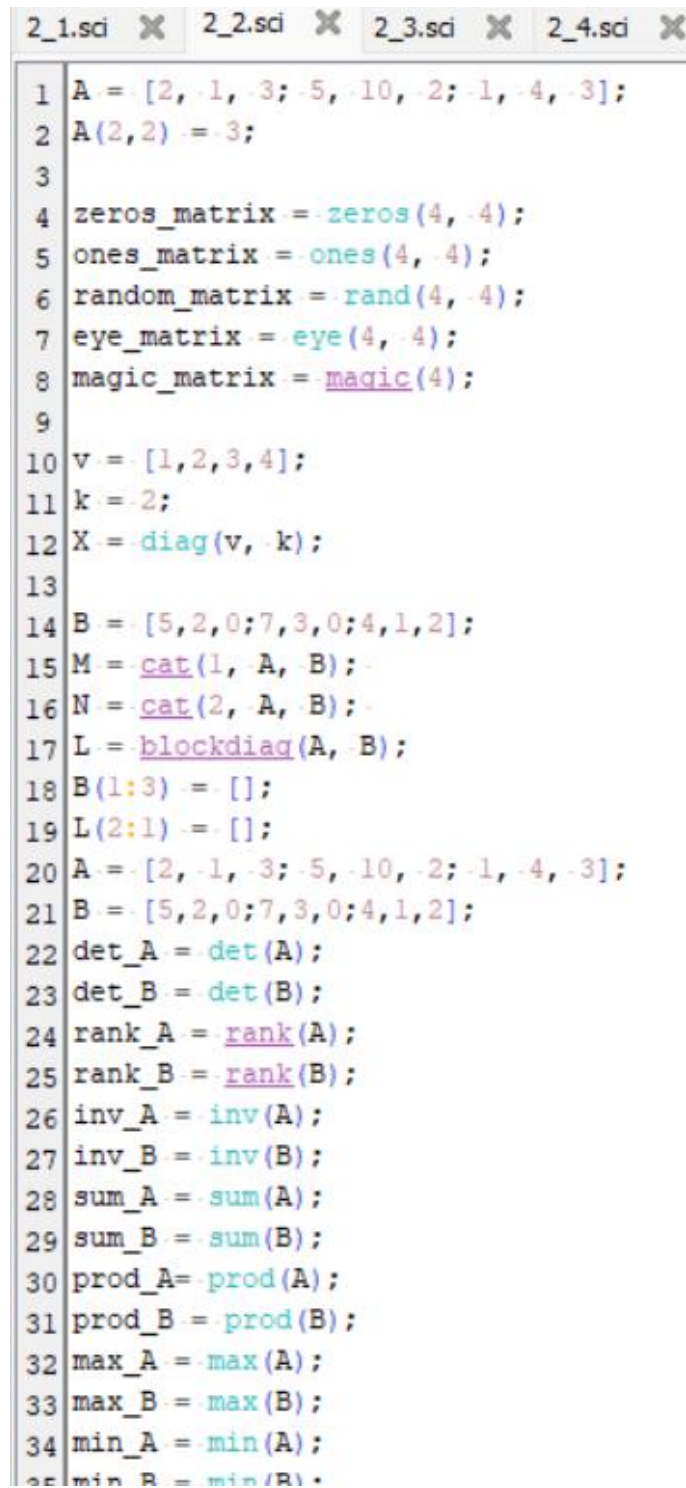
Задание 2. Для вычислений необходимо написать скрипт во встроенном редакторе Scipad. Скриншот варианта представлен на рисунке 3.

-
1. Задать матрицу $A = \begin{pmatrix} 2 & 1 & 3 \\ 5 & 10 & 2 \\ 1 & 4 & 3 \end{pmatrix}$.
 2. Изменить значение элемента матрицы A , используя индексы: $A(2,2)=3$.
 3. Создать квадратные матрицы размерности 4 из нулей, единиц, случайных чисел с нормальным и равномерным распределением, единичную матрицу и магический квадрат.
 4. Задать матрицу X , где вектор v будет k -ой диагональю матрицы.
$$v=(1 \ 2 \ 3 \ 4), \ k=2$$
 5. Объединить в матрицах M , N , L матрицы A и B блочным способом, а также с помощью функций *sysdiag()* и *cat()*.
$$B = \begin{pmatrix} 5 & 2 & 0 \\ 7 & 3 & 0 \end{pmatrix}$$
 6. Удалить из матрицы B третий столбец, из матрицы L – вторую строку.
 7. Посчитать определители и ранги матриц A и B . Найти их обратные матрицы, а также сумму и произведение элементов, максимальный и минимальный элемент.

Рисунок 3 – Скриншот варианта ко второму заданию

Создаём матрицу, меняем значение, создаём квадратные матрицы, в скобках написаны функции в Scilab, предназначенные для реализации задания, из нулей(*zeros*), единиц(*ones*), случайные числа(*rand*), единичную(*eye*), магическую(*magic*). Создаём матрицу X через вектор v и k , создаём матрицы M , N и L через блочный способ, *cat* и *blockdiag*(вместо *sysdiag*). Удаляем столбцы и строки матрицы методом обнуления элементов, то есть замена на пустую строку, столбец. Определители через *det*, ранг через *rank*, обратную

через `inv`, произведение через `prod`, максимальные и элементы найдены. Результат кода представлен на рисунке 4.



```
2_1.sci X 2_2.sci X 2_3.sci X 2_4.sci X
1 A = [2, -1, -3; -5, -10, -2; -1, -4, -3];
2 A(2,2) = -3;
3
4 zeros_matrix = zeros(4, -4);
5 ones_matrix = ones(4, -4);
6 random_matrix = rand(4, -4);
7 eye_matrix = eye(4, -4);
8 magic_matrix = magic(4);
9
10 v = [1, 2, 3, 4];
11 k = -2;
12 X = diag(v, k);
13
14 B = [5, 2, 0; 7, 3, 0; 4, 1, 2];
15 M = cat(1, A, B);
16 N = cat(2, A, B);
17 L = blockdiag(A, B);
18 B(1:3) = [];
19 L(2:1) = [];
20 A = [2, -1, -3; -5, -10, -2; -1, -4, -3];
21 B = [5, 2, 0; 7, 3, 0; 4, 1, 2];
22 det_A = det(A);
23 det_B = det(B);
24 rank_A = rank(A);
25 rank_B = rank(B);
26 inv_A = inv(A);
27 inv_B = inv(B);
28 sum_A = sum(A);
29 sum_B = sum(B);
30 prod_A = prod(A);
31 prod_B = prod(B);
32 max_A = max(A);
33 max_B = max(B);
34 min_A = min(A);
35 min_B = min(B);
```

Рисунок 4 – Скриншот кода к заданию два

Задание 3. Вычислить матрицу, обратную к матрице D. Для вычислений необходимо написать скрипт во встроенном редакторе Scipad. Скриншот варианта представлен на рисунке 5.

11. $D = (A - B)A^2 + 3B$, где

$$A = \begin{pmatrix} 3 & 2 & -5 \\ 4 & 2 & 0 \\ 1 & 1 & 2 \end{pmatrix}, \quad B = \begin{pmatrix} -1 & 2 & 4 \\ 0 & 3 & 2 \\ -1 & -3 & 4 \end{pmatrix}$$

Рисунок 5 – Скриншот варианта к заданию три

Вводим известные данные, переписываем выражение, выводим результат (рисунок 6).

```
2_1.sc X 2_2.sc X 2_3.sc X 2_4.sc X
1 A = [3, 2, -5; 4, 2, 0; 1, 1, 2];
2 B = [-1, 2, 4; 0, 3, 2; -1, -3, 4];
3 D = (A-B)*A^2+3*B;
4 mprintf("----ИСХОДНАЯ МАТРИЦА----\n");
5 disp(D);
6 mprintf("-----ОБРАТНАЯ МАТРИЦА-----\n");
7 disp(inv(D));
```

Рисунок 6 – Скриншот кода к заданию три

Задание 4. Оформить в виде пользовательской функции решение задачи согласно варианту, представленному на рисунке 7.

11	Даны действительные числа s,t. Получить $g(-1.2,s) + g(t,s) - g(2,s*t)$, где $g(a,b) = \frac{\sqrt{a+b} - a^2}{a \cdot b + a - b} + \sqrt{\frac{ a-b }{3}}$
----	---

Рисунок 7 – Скриншот варианта к заданию четыре

Создаём функцию, высчитывающую данное выражение. Задаём известные данные, выводим результат (рисунок 8).

```
2_1.sc X 2_2.sc X 2_3.sc X 2_4.sc X
1 function rez=g(a,b)
2     rez = ((sqrt(a+b)-a^2)/(a*b+a-b))+sqrt(abs(a-b)/3);
3 endfunction
4
5 s = -10;
6 t = -8;
7
8 result = g(-1.2, s) + g(t, s) - g(2, s*t)
9 disp(result);
```

Рисунок 8 – Скриншот кода к заданию четыре

Вывод: получены навыки работы с встроенными и пользовательскими функциями.